

WindowQuant: Mixed-Precision KV Cache Quantization based on Window-Level Similarity for VLMs Inference Optimization

WEI TAO, Huazhong University of Science and Technology, China

XIAOYANG QU, Ping An Technology (Shenzhen) Co., Ltd., China

PEIQIANG WANG, Tsinghua University, Shenzhen International Graduate School, China

GUOKUAN LI, Huazhong University of Science and Technology, China

JIGUANG WAN, Huazhong University of Science and Technology, China

KAI LU, Huazhong University of Science and Technology, China

JIANZONG WANG, Ping An Technology (Shenzhen) Co., Ltd., China

Recently, video language models (VLMs) have been applied in various fields. However, the visual token sequence of the VLM is too long, which may cause intolerant inference latency and GPU memory usage. Existing methods propose mixed-precision quantization to the key-value (KV) cache in VLMs based on token granularity, which is time-consuming in the search process and hardware inefficient during computation. This paper introduces a novel approach called WindowQuant, which employs window-adaptive mixed-precision quantization to optimize the KV cache. WindowQuant consists of two modules: window-level quantization search and window-level KV cache computation. Window-level quantization search quickly determines the optimal bit-width configuration of the KV cache windows based on the similarity scores between the corresponding visual token windows and the text prompt, maintaining the model accuracy. Furthermore, window-level KV cache computation reorders the KV cache windows before quantization, avoiding the hardware inefficiency caused by mixed-precision quantization in inference computation. Extensive experiments demonstrate that WindowQuant outperforms state-of-the-art VLM models and KV cache quantization methods on various datasets.

CCS Concepts: • **Computer systems organization** → **Neural networks**; • **Theory of computation** → **Data compression**; • **Computing methodologies** → **Computer vision tasks**; **Natural language generation**.

Additional Key Words and Phrases: VLM inference, KV cache, window-level quantization search, window-level KV cache computation

ACM Reference Format:

Wei Tao, Xiaoyang Qu, Peiqiang Wang, Guokuan Li, Jiguang Wan, Kai Lu, and Jianzong Wang. 2018. WindowQuant: Mixed-Precision KV Cache Quantization based on Window-Level Similarity for VLMs Inference Optimization. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 25 pages. <https://doi.org/XXXXXXX.XXXXXXX>

Authors' Contact Information: [Wei Tao](#), Huazhong University of Science and Technology, Wuhan, Hubei, China; [Xiaoyang Qu](#), Ping An Technology (Shenzhen) Co., Ltd., Shenzhen, Guangdong, China; [Peiqiang Wang](#), Tsinghua University, Shenzhen International Graduate School, Shenzhen, Guangdong, China; [Guokuan Li](#), Huazhong University of Science and Technology, Wuhan, Hubei, China; [Jiguang Wan](#), Huazhong University of Science and Technology, Wuhan, Hubei, China; [Kai Lu](#), Huazhong University of Science and Technology, Wuhan, Hubei, China; [Jianzong Wang](#), Ping An Technology (Shenzhen) Co., Ltd., Shenzhen, Guangdong, China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

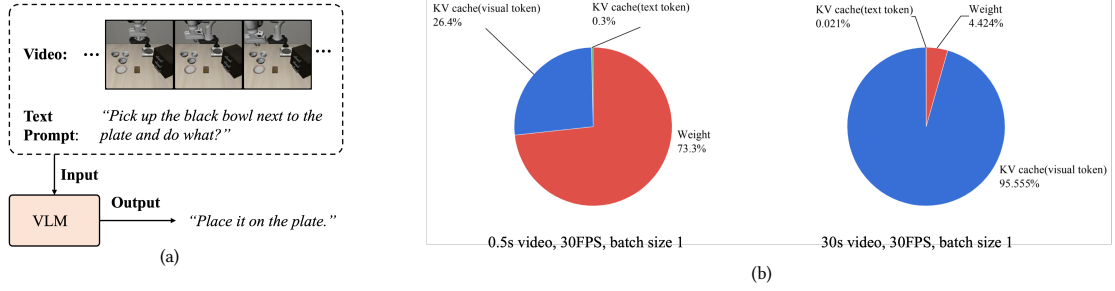


Fig. 1. (a): An example of VLM application. (b): The memory usage comparison of KV cache (including visual token and text token) and weight of the Llava-OneVision-0.5B model when processing two videos of different lengths. Left: 0.5s video. Right: 30s video.

1 Introduction

Large Language Models (LLMs) have gained significant popularity in both academia and industry due to their remarkable capabilities in understanding and generating human-like text. Recently, there has been a growing interest in extending LLMs to handle multimodal data, particularly video. Generally, when LLMs are used for video understanding, the input video is first converted into a sequence of frames, and each frame in the sequence is then transformed into tokens by spatiotemporal visual encoders, referred to as **visual tokens** (also called visual embeddings). Additionally, a piece of text is provided as a prompt to guide the LLM, and this text is also converted into tokens by text encoders, referred to as **text tokens** (also called text embeddings). The visual and text tokens are concatenated and fed into the LLM for processing. Therefore, LLMs for video understanding are also called Vision-Language Models (VLMs). VLMs have been applied to various fields such as video captioning, temporal grounding, action recognition, and embodied intelligence. Figure 1(a) demonstrates a typical example of a VLM application, where the VLM generates the answer to the question in the input text prompt based on the input video and the clue in the text prompt. By leveraging pretraining on large-scale video-text datasets, VLMs offer a powerful framework for reasoning about complex temporal dynamics and semantic content within videos.

However, current VLMs often face issues of insufficient GPU memory and excessively high inference latency when performing video understanding tasks. These challenges are mainly caused by the visual tokens' key-value (KV) cache. The memory footprint of a VLM's weights and text prompts remains static, but as the input video duration increases, the growing number of visual tokens leads to correspondingly larger KV cache memory consumption. For example, in the LLaVA-OneVision-0.5B model, the memory usage of the model weight is about 1GB. Generally, a text prompt contains dozens of words and will be transformed into several dozen text tokens, the memory usage of which is about 4MB. However, a single frame image will be converted into 256 visual tokens. Therefore, for a 30-second video at 30 FPS, even with a batch size of just 1 for inference, the memory consumption of the KV cache of visual tokens can reach an astonishing 21.6GB, easily exceeding the memory limits of typical GPUs. As illustrated in Figure 1(b), when the input video is very short (only 0.5s), the model weight still dominates the memory usage. However, when the input video is a little longer (30s), the memory bottleneck shifts to the visual tokens' KV cache. The large KV cache not only leads to GPU memory insufficiency but also significantly increases inference latency. Since we need to transfer the KV cache data between GPU memory and the high-speed static random-access memory (SRAM) during inference, such a massive KV cache leads to substantial data transmission latency, ultimately increasing overall inference latency. Figure

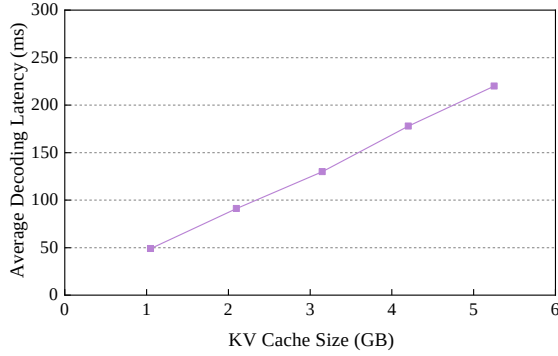


Fig. 2. The average decoding latency of LLaVA-OneVision-Qwen2-7B model with different KV cache size.

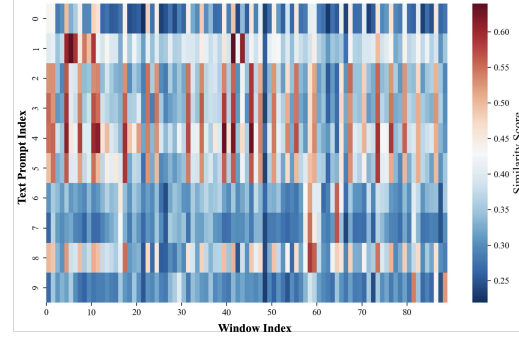


Fig. 3. The similarity heatmap between a video and 10 text prompts. Most of the video windows are irrelevant to the text prompt.

2 shows the variation of the average decoding latency required to generate one token for LLaVA-OneVision-Qwen2-7B under different KV cache sizes. As can be observed from the figure, the average decoding latency increases almost linearly with the growth of the KV cache size.

Researchers have proposed various methods to compress the KV cache, such as attention pruning [1, 3, 34], removing or quantizing unimportant tokens [15, 43, 48], and considering optimizations from a system perspective [7, 8, 19]. Among them, the simplest and most effective approach is quantization. Quantization means converting the KV cache from the original FP16 type to a lower bit-width type (such as INT8, INT4, or even INT2). However, most existing KV cache quantization efforts [24, 28, 35, 49] uniformly quantize the KV cache to a single bit-width. In reality, the importance of tokens within the KV cache is hierarchical, meaning some tokens are more critical than others. Therefore, uniform quantization can easily cause accuracy degradation. Some papers [11, 16, 17, 46] have proposed implementing mixed-precision quantization based on the importance of the tokens to address this issue, but their quantization search strategies are token-level and very time-consuming. Even worse, mixed-precision quantization can cause hardware inefficiency during the inference computation, such as increased latency and memory usage.

In this paper, we propose to apply the mixed-precision quantization to the KV cache at a larger granularity, i.e., the **window level**. We observed an interesting phenomenon: In the VLM inference process, not all visual tokens are relevant to the text prompt, and there is a significant amount of redundant information. We select a long video (which can be split into 89 windows) to simulate a long context and create 10 text prompts, calculating their similarity. As shown in Figure 3, for each text prompt, there is only a small portion of windows that are highly relevant, while most are irrelevant. Since text prompts serve as the guiding instructions for VLMs, visual information unrelated to the text prompts can be regarded as contributing little to the task itself. Based on the above observation, we need to retain precision for the KV cache corresponding to the visual token windows that are highly relevant to the text prompt; otherwise, the model would lose critical information. For the KV cache corresponding to visual token windows irrelevant to the text prompt, quantizing them to very low bits will not significantly affect the model’s accuracy. Given these observations, we propose a window-adaptive mixed-precision KV cache quantization method called **WindowQuant**.

WindowQuant mainly contains two modules: window-level quantization search and window-level KV cache computation. Window-level quantization search calculates the similarity scores between the text prompt and each visual token window and determines the bit-width configuration of the KV cache windows corresponding to the visual

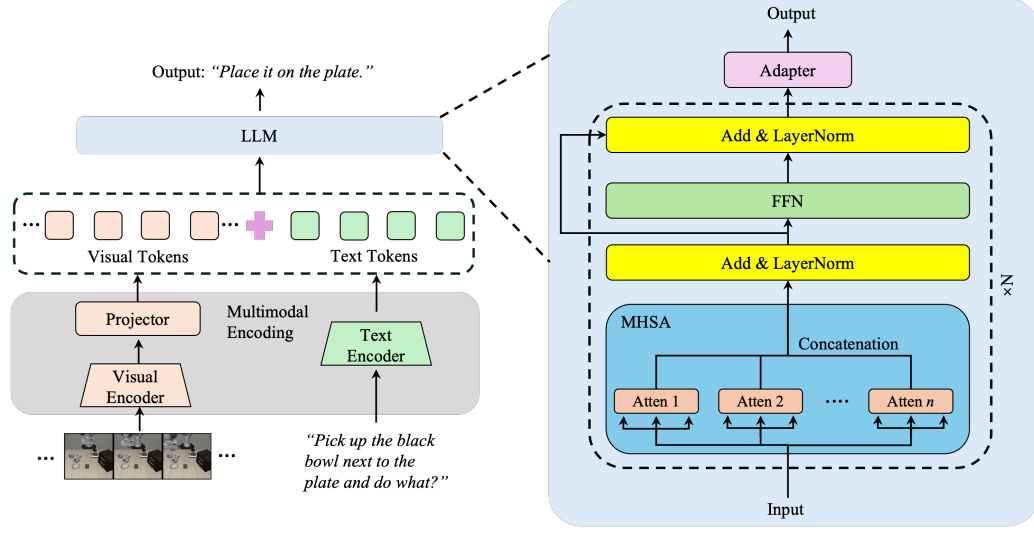


Fig. 4. The architecture of typical VLMs.

token windows according to those scores. This module maintains the accuracy of VLMs quickly and effectively. Since our quantization configuration is determined from the window perspective—rather than assigning it token by token as in previous methods—our configuration search time is much shorter. Moreover, the subsequent quantization is also performed on a per-window basis, with all tokens in a window quantized together, which further reduces the overall quantization time. We further design the window-level KV cache computation module to avoid hardware inefficiency. Window-level KV cache computation reorders the context KV cache windows to ensure the same bit-width configuration in which KV cache windows are arranged contiguously in physical memory. This module significantly optimizes the inference latency and memory usage during VLM inference.

In summary, our contributions can be outlined as follows:

- We propose a window-adaptive mixed-precision KV cache quantization method called WindowQuant for effective and efficient VLM inference.
- WindowQuant contains the window-level quantization search module, which determines the bit-width configuration of KV cache windows based on the similarity scores between the corresponding visual token windows and the text prompt, maintaining the inference accuracy of the VLM.
- WindowQuant contains the window-level KV cache computation module, which reorders KV cache windows in different bit-width configurations before quantization, avoiding the issue of hardware inefficiency during inference caused by mixed-precision quantization.
- Extensive experiments validate that WindowQuant achieves better model accuracy, higher decoding throughput, and reduced memory usage compared with state-of-the-art (SOTA) methods.

2 Preliminary

In this section, we briefly review the necessary background and preliminaries to facilitate understanding of the proposed method, especially for readers who may be less familiar with vision–language model inference systems. The content

presented here primarily summarizes established designs and observations from prior work, rather than introducing new technical contributions. The main content of this section is derived from the paper “*Attention Is All You Need*” [39].

2.1 VLM Architecture

Figure 4 illustrates a typical VLM architecture [25]. It primarily consists of a visual encoder, a text encoder, and an LLM. The input video and text prompt first go through a multimodal encoding stage. The video is transformed into visual tokens (also called visual embeddings) by the visual encoder and the projector (for modality alignment), while the text prompt is converted into text tokens (also called text embeddings) by the text encoder. These two kinds of tokens are then concatenated to form the input embeddings and fed into the LLM, which processes them to produce the output. Notably, both the visual encoder and text encoder are relatively lightweight, with the majority of computations occurring within the LLM, where the KV cache is also located. Therefore, our primary focus is on optimizing the LLM’s computations, and to this end, we have also detailed the LLM’s structural components.

As illustrated in Figure 4, the standard LLM architecture is composed of multiple blocks stacked sequentially [39]. Each block typically consists of three core components: a Multi-Head Self-Attention (MHSA) module, a Feed-Forward Network (FFN) module, and a Layer Normalization module. These components operate in a cascaded manner, where each block takes the output features from the preceding block as input and applies transformations through its internal sub-modules.

The core of the LLM architecture is the self-attention mechanism employed in the MHSA block. Given a vector of input embeddings $\mathbf{X} = [x_1, x_2, \dots, x_n]$, the MHSA module applies linear projections using a query weight matrix $\mathbf{W}^Q$, key weight matrix $\mathbf{W}^K$, and value weight matrix $\mathbf{W}^V$ to compute the query ($\mathbf{Q}$), key ($\mathbf{K}$), and value ($\mathbf{V}$) matrices. The calculation is illustrated in Equation 1:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \mathbf{K} = \mathbf{X}\mathbf{W}^K, \mathbf{V} = \mathbf{X}\mathbf{W}^V \quad (1)$$

The tuple $(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ undergoes a self-attention operation, where the attention weights $\mathbf{A}$ are first computed by applying the text prompt matrix $\mathbf{Q}$ and the key matrix $\mathbf{K}$, as shown in Equation 2:

$$\mathbf{A} = \text{Attention}(\mathbf{Q}, \mathbf{K}) = \frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} \quad (2)$$

Here, d_k denotes the dimensionality of the token embeddings. The attention weights are then processed and multiplied by the value matrix $\mathbf{V}_i$ to obtain the output feature $\mathbf{Z}_i$, as shown in Equation 3:

$$\mathbf{Z}_i = \text{Softmax}(\mathbf{A}_i + \mathbf{M})\mathbf{V}_i \quad (3)$$

where the matrix $\mathbf{M}$ represents the mask matrix, which is used to ensure that the attention weights for tokens appearing after the current token in the sequence are set to zero. The mask matrix is only applied in the prefill stage. In the decoding stage, there is no mask matrix. We will introduce the prefill stage and decoding stage later.

2.2 LLM Inference Process

Figure 5 illustrates the inference process of a typical pre-trained LLM in the VLM, where the blue boxes represent each output token. The inference process can be divided into two main stages:

- **Prefill Stage:** During the prefill stage, the LLM receives the visual tokens and text tokens generated by the multimodal encoding process of the VLM. Then the VLM concatenates the visual tokens and text tokens and

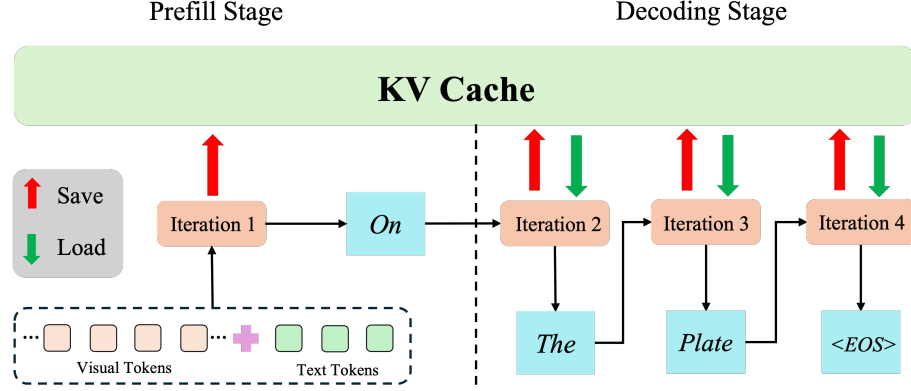


Fig. 5. The inference process of typical LLMs.

feeds them together into a pre-trained LLM. The pre-trained LLM calculates the input tokens in parallel and generates the first output token.

- **Decoding Stage:** During the decoding stage, the pre-trained LLM in the VLM combines the entire input token sequence with the already generated output tokens to produce the next token in the output sequence. This process iterates until the end-of-sentence token (<EOS>) is generated.

The primary computational load in the aforementioned VLM inference process lies in the self-attention calculation mentioned in the previous section. Since the decoding stage is iterative and cannot be parallelized, it introduces the first major challenge in large model inference: each decoding step requires recomputing the K and V matrices for the entire input token sequence and the already generated output tokens, leading to significant redundant computation and severely increasing inference latency. To address this issue, researchers developed **KV cache**, which stores the K and V matrices of both the input tokens and the already generated output tokens in GPU memory. When computation is needed, these matrices are loaded from GPU memory into the GPU’s high-speed SRAM. After computing the K and V matrices for a new output token, they are stored in the GPU memory and appended to the end of the existing KV cache.

However, as discussed in Section 1, an excessively large KV cache leads to a significant increase in GPU memory consumption and inference latency. Therefore, how to optimize the size of the KV cache has become a critical issue for VLM optimization. In the following, we introduce how to leverage mixed-precision quantization to reduce the KV cache size.

3 Method

In this section, we first outline the overall architecture of our WindowQuant approach. Next, we provide a detailed description of the two main components of WindowQuant, namely window-level quantization search and window-level KV cache computation.

3.1 Architecture Overview

The architecture of WindowQuant is shown in Figure 6. The input video is first segmented into several equal-length, short windows (If the length of the video is not divisible by the window size, we truncate the portion at the end of the video that cannot be divided by the window size. The KV cache of this portion will be kept in FP16 precision).

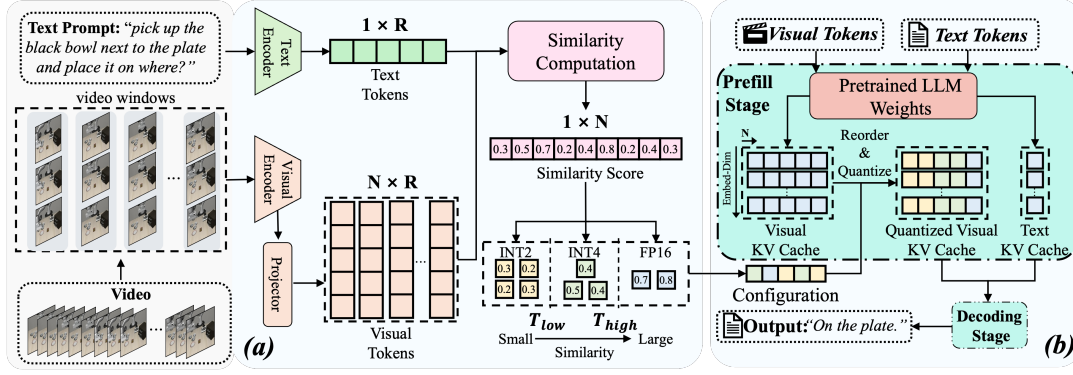


Fig. 6. The architecture overview of WindowQuant. (a) The window-level quantization search module. (b) The window-level KV cache computation module.

The text prompt is converted into text tokens by the text encoder, while the video windows are converted into visual token windows by the visual encoder and the projector. Then, the window-level quantization search receives and processes the visual token windows and the text prompt, outputting the quantization bit-width configuration. Next, the visual token windows and the text tokens are fed into the pre-trained LLM for formal inference. During the prefill stage, we reorder the KV cache windows generated from the visual token windows. The reordered KV cache windows output from this module are subsequently quantized (all tokens in a window are quantized together) according to the configuration provided by the window-level quantization search module. Finally, after the decoding stage, the pre-trained LLM outputs the answer. To maintain model accuracy, we only quantize the KV cache of the input visual tokens while retaining FP16 precision for the KV cache of the output tokens and the text tokens. Given that, in general VLM inference tasks, the length of the visual tokens is significantly larger than that of the output tokens plus text tokens, we believe this strategy will not result in significant memory or inference latency overhead.

3.2 Window-level Quantization Search

3.2.1 Quantization Configuration Determination. Window-level quantization search draws inspiration from the concept of retrieval augmentation generation (RAG). RAG retrieves relevant documents or passages from an external knowledge source (such as a database or corpus) to augment the information contained in the current LLM input text prompt, thereby achieving better generation results. In our method, we find the video windows that are relevant to the text prompt and only retain the KV cache corresponding to them. To determine relevance, we directly use the visual token windows and the text tokens to calculate similarity.

As shown in Figure 6, video windows and text prompt are sent separately to the visual encoder and text encoder. After encoding and aligning modalities, we obtain the visual token windows and text tokens. Visual tokens in a window are essentially a subset of the original VLM visual tokens. Specifically, they consist of the tokens produced by the visual encoder from the video frames within the corresponding video window. We denote the original visual token matrix as

$$\mathbf{V} = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_M\} \in \mathbb{R}^{M \times D} \quad (4)$$

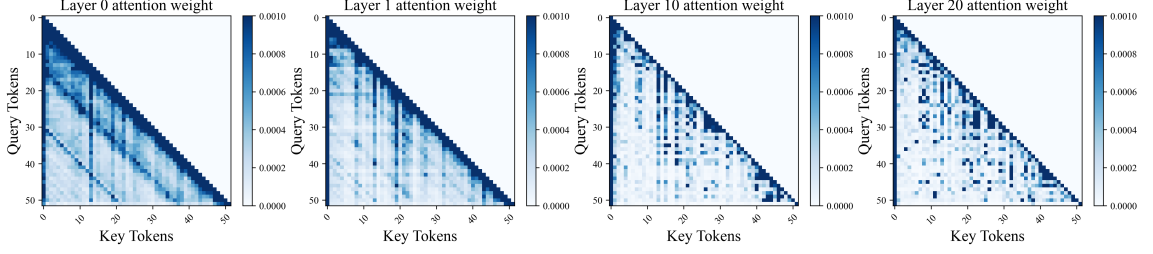


Fig. 7. The attention weights of the tokens in four different layers in the pre-trained LLM of the Llava-Onevision-Qwen2-7B model.

where M is the number of visual tokens, D is the dimension of encoder and $\mathbf{v}_i \in \mathbb{R}^D$ denotes the vector of the i -th visual token. For a video input consisting of T frames, the total number of visual tokens is

$$M = T \times K \quad (5)$$

where K denotes the number of visual tokens per frame, which is typically set to 256 or 576 depending on the input resolution and the configuration of the visual encoder.

Similarly, we denote the text token matrix as

$$\mathbf{T} = \{\mathbf{t}_1, \mathbf{t}_2, \dots, \mathbf{t}_N\} \in \mathbb{R}^{N \times D} \quad (6)$$

where N is the number of text tokens and $\mathbf{t}_j \in \mathbb{R}^D$ denotes the vector of the j -th text token. Here, D denotes the embedding dimension.

Suppose the window size is S , then the visual tokens matrix of the i -th window can be denoted as

$$\mathbf{W}_i = \{\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_S\} = \{\mathbf{v}_{i \times S}, \mathbf{v}_{i \times S + 1}, \dots, \mathbf{v}_{i \times S + S - 1}\} \in \mathbb{R}^{S \times D} \quad (7)$$

where $\mathbf{w}_k \in \mathbb{R}^D$ denotes the vector of the j -th visual token in the window and there are $\lfloor \frac{M}{S} \rfloor$ windows in total.

Next, we calculate the similarity between the text token matrix and each visual token window matrix, resulting in a similarity score list. The similarity formula is:

$$\text{sim}(\mathbf{T}, \mathbf{W}_i) = \frac{1}{SN} \sum_{j=1}^N \sum_{k=1}^S \frac{\mathbf{t}_j \cdot \mathbf{w}_k}{\|\mathbf{t}_j\| \times \|\mathbf{w}_k\|} \quad i = 1, 2, \dots, \lfloor \frac{M}{S} \rfloor \quad (8)$$

where $\|\cdot\|$ means the L2 norm.

Subsequently, we set two thresholds, T_{low} and T_{high} ($0 < T_{low} < T_{high} < 1$). We compare the similarity score at each index in the similarity score list with these two thresholds. For a similarity score greater than T_{high} , the visual token window at that index is considered highly similar to the text tokens, and the video window at that index is considered highly relevant to the text prompt, and we set the bit-width configuration of its corresponding KV cache window as FP16. For a similarity score less than T_{low} , the video window at that index is considered to have little relevance to the text prompt, and we set the bit-width configuration of its corresponding KV cache window as INT2. For a similarity score between T_{low} and T_{high} , we adopt a compromise strategy, setting the bit-width configuration of its corresponding KV cache window as INT4. We will introduce how we set these two thresholds later.

3.2.2 Fixed Precision of First Window. Previous researchers [43] have demonstrated that the token at the initial position of an LLM plays a crucial role in the model's performance, significantly influencing its accuracy. In our research, we

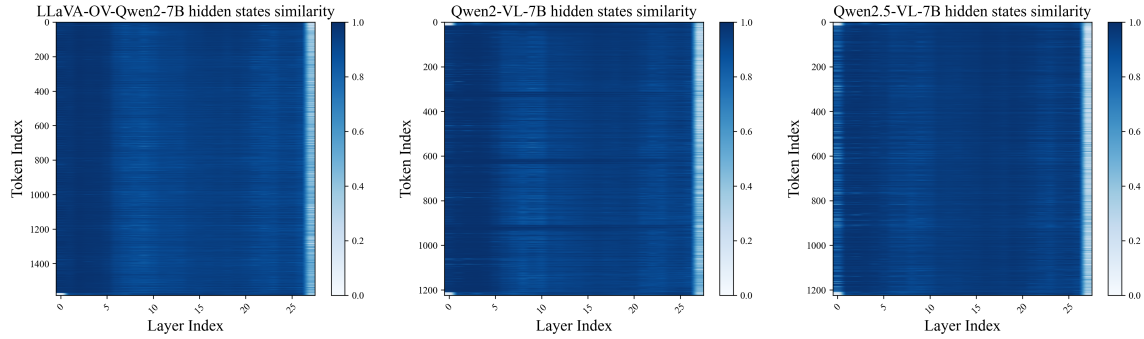


Fig. 8. The hidden states similarity between layers, results are tested on three pre-trained LLMs of different VLM models. From left to right, the corresponding VLMs are LLaVA-QneVision-Qwen2-7B, Qwen2-VL-7B, and Qwen2.5-VL-7B (where “OV” in the figure denotes OneVision).

further explore this phenomenon in VLMs. We conduct an experiment to depict the attention weights of the initial few visual tokens of different layers within the pre-trained LLM of Llava-OneVision-Qwen2-7B model (attention weights are the “ A_i ” in Equation 2, which can reflect the contribution of a token to model inference accuracy). As depicted in Figure 7, we observe that the attention weights for visual tokens at the initial positions are relatively higher than those for tokens in subsequent positions. This finding strongly suggests that visual tokens at the beginning of the sequence are highly influential, playing a critical role in determining the model’s output. These initial visual tokens seem to capture essential contextual information, which is then propagated through the rest of the sequence.

In response to these observations, we propose to fix the precision of the first visual token window to FP16, regardless of its similarity score. This approach guarantees that the important information carried by the visual tokens at the start of the sequence won’t be lost, thus protecting the model’s accuracy.

3.2.3 Threshold Determination. Note that LLMs contain numerous layers, and different layers may exhibit varying sensitivities to quantization. To investigate this, we follow the idea of SqueezeAttention [41], using the cosine similarity between the hidden states before and after layer attention to represent the contribution of a layer to the model accuracy. Specifically, the similarity is calculated as follows:

$$s_i = \frac{h_i \cdot h'_i}{||h_i|| \cdot ||h'_i||} \quad (9)$$

where h_i is the hidden states vector before the attention computation of layer i and h'_i is the hidden states vector after the attention computation of layer i .

A large s_i indicates that the hidden states vector has not changed much when passing through the i_{th} layer, which means that the contribution of this layer to the model accuracy is not high.

We depict the hidden states’ similarity between layers on three pre-trained LLMs of different VLM models, and the results are shown in Figure 8. In the figure, colors closer to white indicate lower similarity, while colors closer to blue indicate higher similarity. From the figure, we observe that across all three models, most layers exhibit high hidden states similarity, while only few final layers have low hidden states similarity. Therefore, most of the layers in the VLM have little contribution to the model accuracy. Only few layers are quite important to the model accuracy. Thus, we can adopt different quantization strategies for different layers in the VLM.

Specifically, we adopt the strategy of quantizing as many KV cache values as possible to INT2 for layers with large s_i . That is, we set both T_{low} and T_{high} relatively high, close to 1. Accordingly, we believe that layers with smaller s_i contribute more to the model accuracy and are therefore not suitable for quantization. For these layers, we adopt the strategy of keeping as many KV cache values as possible in FP16 and setting both T_{low} and T_{high} relatively low, close to 0. For the other layers, we will quantize the values in the KV cache to INT4 as much as possible, meaning we will set a small T_{low} close to 0 and a large T_{high} close to 1. Finally, we design these two threshold functions to calculate the T_{low} and T_{high} of layer i :

$$T_{low}^i = f_1(s_i) = \frac{e^{\alpha s_i} - 1}{e^{\alpha} - 1} \quad (10)$$

$$T_{high}^i = f_2(s_i) = \frac{e^{-\alpha s_i} - 1}{e^{-\alpha} - 1} \quad (11)$$

where α is a pre-defined hyperparameter that controls the threshold. These two functions perfectly meet the aforementioned requirements. When s_i approaches 1, both $f_1(s_i)$ and $f_2(s_i)$ approach 1; when s_i approaches 0, both $f_1(s_i)$ and $f_2(s_i)$ approach 0. In other cases, $f_1(s_i)$ tends to be closer to 0 while $f_2(s_i)$ tends to be closer to 1. Moreover, within the interval $[0, 1]$, $f_1(s_i)$ is always less than or equal to $f_2(s_i)$, perfectly satisfying the requirements for T_{low} and T_{high} .

The overall algorithm of our window-level quantization search is shown in Algorithm 1.

For batch-arriving requests, to preserve the convenience of batch computation, we apply the same quantization configuration to all requests within a batch. The procedure is as follows. First, for each request, T_{low} and T_{high} are identical, since they depend only on the current layer index and the hyperparameter α . Second, for a given window position, we compute the similarity at that window for all requests in the batch, and determine the corresponding quantization configuration by comparing the similarity with T_{low} and T_{high} . The quantization configuration that occurs most frequently is then selected and applied to that window for all requests in the batch. This process is repeated to determine the quantization configuration for all windows.

3.3 Window-level KV Cache Computation

3.3.1 KV Cache Window Reordering. Directly applying mixed-precision quantization can result in visual token windows with different bit-widths that are physically contiguous, which can lead to hardware inefficiency during inference computation. For example, modern GPUs use cache lines to optimize data reading, but data with different bit-widths cannot be aligned properly, potentially spanning multiple cache lines. This requires loading more cache lines, increasing the cache miss rate and memory access frequency, which significantly raises inference latency. Furthermore, if the hardware is designed to read multi-byte data simultaneously, such as single instruction multiple data (SIMD) architecture, some of the narrower bit-width data may only use part of the hardware resources, leading to GPU memory waste.

Therefore, we design the window-level KV cache computation module to address the hardware inefficiency problem. The algorithm pseudocode is shown in Algorithm 2 (For writing convenience, we omit the division by the scalar factor $\sqrt{d_k}$ in Equation 2). Specifically, during the prefill stage, we first calculate the output token. Then we adopt KV cache window reordering. Then, the KV cache windows with the same bit-width configuration are arranged together, making them contiguous in physical memory. Next, these windows are quantized following the bit-width configuration determined by the KV cache quantization search module. During the decoding stage, we multiply the Q matrix by the transposed three blocks of the K matrix with three different bit-widths to obtain three attention matrix blocks. These blocks are then concatenated along the last dimension to form the attention matrix. The attention matrix is then added to the mask matrix and processed with the softmax function. The processed attention matrix is divided into three blocks

Algorithm 1: Pseudocode of window-level quantization search

Input: Threshold parameter a , VLM model M , window size S , VLM visual encoder VE , VLM text encoder TE , video dataset V , text prompt T , Number of model layers N

Output: Quantization bit-width configuration C

Sample a calibration dataset $V' = \text{sample}(V)$;

Run a prior inference with standard flash attention on the calibration dataset and achieve hidden states $H = M(V' + T)$;

Initialize two threshold sets T_{low}, T_{high} ;

for $i = 0$ **to** $N - 1$ **do**

$s_i = \frac{H_i \cdot H_{i+1}}{||H_i|| \cdot ||H_{i+1}||}$;
 $T_{low}^i = \frac{e^{as_i} - 1}{e^a - 1}$;
 $T_{high}^i = \frac{e^{-as_i} - 1}{e^{-a} - 1}$;

end

Achieve visual embedding $E_v = VE(D)$ and text embedding $E_t = VT(T)$;

Split visual embedding into windows $E_w = \text{split}(E_v, S)$;

for $i = 0$ **to** $N - 1$ **do**

for $j = 0$ **to** $\text{len}(E_w) - 1$ **do**

$\text{sim}_j^i = \frac{E_w^i \cdot E_t}{||E_w^i|| \cdot ||E_t||}$;
if $\text{sim}_j^i > T_{high}^i$ **then**

$C_j^i = 16$

end
else if $\text{sim}_j^i < T_{low}^i$ **then**

$C_j^i = 2$

end
else

$C_j^i = 4$

end

end

end

Return C ;

again, and each is multiplied by the V matrix blocks with the corresponding bit-width to produce three small output matrices. The final output matrix is obtained by summing these three small output matrices.

3.3.2 Correctness Proof. We prove that the output obtained in this way is the same as the output from the traditional computation method. In traditional computation method, assuming there are N visual token windows, then K matrix can be divided into a block matrix of the form $[K_1|K_2|\dots|K_N]^T$, and V matrix can be divided into a block matrix of the form $[V_1|V_2|\dots|V_N]^T$, both of which are partitioned along the token length dimension. Obviously, attention matrix A can also be divided into $[A_1|A_2|\dots|A_N]$. According to block matrix multiplication, the final output matrix O is:

$$O = [A_1V_1 + A_2V_2 + \dots + A_NV_N] \quad (12)$$

Now, if we perform KV cache window reordering, it is equivalent to changing the order of the sub-blocks within the K and V matrices. Suppose after reordering, K becomes $[K_{x_1}|K_{x_2}|\dots|K_{x_N}]^T$, and V becomes $[V_{x_1}|V_{x_2}|\dots|V_{x_N}]^T$, where $\{x_1, x_2, \dots, x_N\}$ is a permutation of $\{1, 2, \dots, N\}$. Since the softmax function is not affected by the order of the matrix

Algorithm 2: Pseudocode of window-level KV cache Computation in a Pytorch-like Style

```

# s: the similarity score list
# N: the number of visual token windows
# K, V: the K, V cache of the visual tokens
# T_low, T_high: the two thresholds
# During the prefill stage:
output = mm(softmax(mm(Q, K)), V)
for i in range(N):
    if s[i] < T_low:
        K_int2 = torch.cat(K_int2, K[i])
        V_int2 = torch.cat(V_int2, V[i])
    elif s[i] > T_high:
        K_fp16 = torch.cat(K_fp16, K[i])
        V_fp16 = torch.cat(V_fp16, V[i])
    else:
        K_int4 = torch.cat(K_int4, K[i])
        V_int4 = torch.cat(V_int4, V[i])
K_q2, K_q4 = quant(K_int2), quant(K_int4)
V_q2, V_q4 = quant(V_int2), quant(V_int4)
# Q: the Q vector of the current token
# len_2, len_4 = len(K_q2), len(K_q4)
# During the decoding stage:
att = fqm(Q, transpose(K_q2), 2)
att = torch.cat(att, fqm(Q, transpose(K_q4), 4), -1)
att = torch.cat(att, mm(Q, transpose(K_fp16)), -1)
att = softmax(att)
output = fqm(att[:len_2], V_q2)
output += fqm(att[len_2:len_2+len_4], V_q4)
output += mm(att[len_2+len_4:], V_fp16)

```

quant: the quantization function; fqm: FP16 matrix and quantized matrix multiply; mm: FP16 matrix multiply; cat: concatenation.

sub-blocks, the softmax result equals the result obtained by first applying softmax to the original attention weights matrix and then performing reordering. That is, sub-blocks of matrix A will eventually be arranged in the same order as the sub-blocks within K , which means A is $[A_{x_1} | A_{x_2} | \dots | A_{x_N}]$. The final output O' will be:

$$O' = [A_{x_1} V_{x_1} + A_{x_2} V_{x_2} + \dots + A_{x_N} V_{x_N}] \quad (13)$$

which is equal to the matrix O due to the commutative invariance of matrix addition.

Figure 9 illustrates an example of the impact of KV cache window reordering on attention computation, where the number of windows is six. From the figure, we can see that, the result before reordering is $A_1 V_1 + A_2 V_2 + A_3 V_3 + A_4 V_4 + A_5 V_5 + A_6 V_6$, while the result after reordering is $A_1 V_1 + A_3 V_3 + A_2 V_2 + A_4 V_4 + A_3 V_3 + A_6 V_6$. Obviously, the attention computation result remains consistent before and after the KV cache window reordering.

3.3.3 Implementation of Quantization. We employ asymmetric round-to-nearest (RTN) quantization in our method. RTN quantization is a method that converts high-precision floating-point numbers into the closest representable integer in the target bit-width. Asymmetric quantization allows the quantized range to be offset (unlike symmetric quantization, where the range is centered around zero), enabling more precise representation of data with non-zero-centered or skewed distributions, which is well-suited for KV cache quantization. The formulas of the quantization and dequantization processes are as follows:

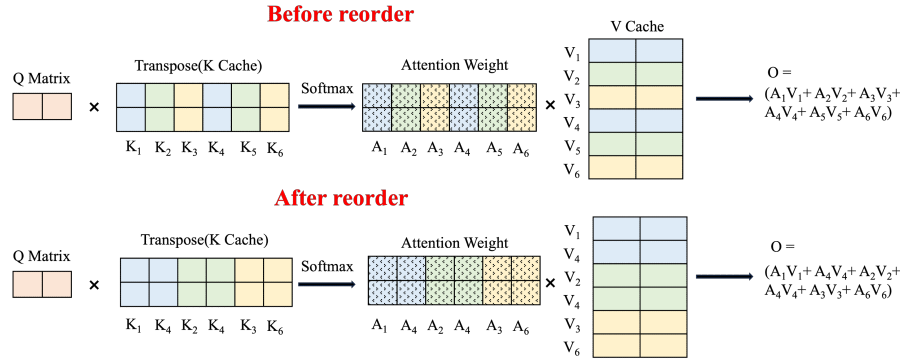


Fig. 9. An example of KV cache window reordering.

Quantization:

$$Q = \text{clamp}(\lfloor \frac{x}{s} \rfloor, q_{\min}, q_{\max}) \quad (14)$$

Dequantization:

$$X = s \cdot (x - z) \quad (15)$$

where Q is the quantized tensor, x is the original precision tensor, X is the dequantized tensor, s is the scaling factor, z is the zero point, $q_{\min}$ and $q_{\max}$ are the integer range bounds ($[0, 2^b - 1]$ for b -bit unsigned quantization and $[-2^{b-1}, 2^{b-1} - 1]$ for b -bit signed quantization), $\lfloor \cdot \rfloor$ is the round-to-nearest operator. s and z are calculated as follows:

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad z = q_{\min} - \lfloor \frac{x}{s} \rfloor \quad (16)$$

where $x_{\max}$ and $x_{\min}$ are the max/min values in the original data.

We apply group-wise quantization to the KV cache, where each group's size corresponds to one window. The idea of group-wise quantization is derived from GPTQ [13], which means all elements within the same group share uniform quantization parameters (i.e., scaling factor and zero point). Different regions of a tensor may have diverse numerical ranges (e.g., outliers in LLM activations). Group-wise quantization adapts to these local variations, reducing quantization error compared to per-tensor methods.

During inference, since the calculation of the softmax function needs high-precision data, we need to dequantize the attention weights matrix after getting it through matrix multiplication. Both the dequantization and matrix multiplication operators require loading matrices from the GPU global memory into the GPU high-speed SRAM, then writing results back to global memory after calculation. Such two operators will cause two read and write operations. To further accelerate the inference process, we apply operator fusion, which fuses the operators of dequantization and matrix multiplication and designs a new operator. The idea of operator fusion is derived from AutoTVM [4]. By fusing these two operators into one, we reduce memory transactions to a single read and write operation, while keeping all intermediate computations within the GPU's high-speed SRAM hierarchy. During the practical inference, we directly input the quantized key matrix and the original precision Q matrix into the new operator, which returns a high-precision floating-point attention weights result for the following softmax function. After the softmax function, we again input the quantized value matrix and the original precision attention weights matrix into the new operator and achieve the high precision attention output. By fusing different operators together, we avoid costly intermediate memory read

Table 1. Performance comparison of various models on multi-choice video QA benchmarks. Some of the results are cited from TS-LLaVA [33].

Model	LLM Size	VisionEncoder	NExT-QA	EgoSchema	IntentQA
Video-LLaVA	7B	ViT-L	60.5	37.0	-
Video-LLaMA2	7B	CLIP-L	-	51.7	-
MovieChat+	7B	CLIP-G	54.8	56.4	-
Vista-LLaMA	7B	CLIP-G	60.7	-	-
DeepStack-L	7B	CLIP-L	61.0	38.4	-
M ³	7B	CLIP-L	63.1	36.8	58.8
IG-VLM	7B	CLIP-L	63.1	35.8	60.3
SF-LLaVA	7B	CLIP-L	64.2	47.2	60.1
TS-LLaVA	7B	CLIP-L	66.5	50.2	61.7
LLaVA-OneVision-Qwen2	7B	SigLIP	67.8	59.8	64.9
LLaVA-OneVision-Qwen2+WindowQuant	7B	SigLIP	78.9	64.1	69.4
Qwen2-VL	7B	ViT-L	75.2	54.6	88.6
Qwen2-VL+WindowQuant	7B	ViT-L	74.6	53.2	87.8
InternVL2	8B	InternViT	68.7	48.8	67.5
InternVL2+WindowQuant	8B	InternViT	76.1	55.8	75.9

and write of dequantized matrices, significantly reduce global memory bandwidth pressure, eliminate kernel launch overhead between operations, and enable hardware-friendly computation patterns that maximize cache utilization and inference latency reduction.

3.3.4 Limitation. It should be noted that our reordering method still has one limitation: reordering will affect the output result in the prefill stage computation. This is because, after the Q and K matrices are multiplied, the attention weights matrix is produced in the prefill stage, and a mask matrix needs to be added to ensure causal relationships in token generation. The presence of this mask matrix causes the reordered attention weights matrix to yield different results when fed into the softmax function. In this case, the softmax output no longer equals the result obtained by first applying softmax to the original attention weights matrix and then performing reordering. In the decoding stage, reordering the KV cache does not affect the output results since no mask matrix is applied in this stage.

Therefore, during actual inference, we first compute the results for the first output token in the prefill stage. We then perform reordering and quantization to obtain the reordered, quantized KV cache (storing only this KV cache without recomputing the first output token). This reordered quantized KV cache subsequently participates in all following computations. In the later decoding stage, we can normally perform reordering before computing each output token.

4 Evaluation

4.1 Experiment Setup

Benchmarks. We begin our evaluation by conducting a comprehensive comparison of WindowQuant’s question-answering (QA) capabilities against several SOTA VLMs across multiple standardized QA benchmarks. Our assessment encompasses three widely-recognized multi-choice QA datasets: NExT-QA, which focuses on temporal and causal reasoning in video understanding; EgoSchema, designed to evaluate long-form video comprehension from an egocentric perspective; and IntentQA, which tests models’ ability to infer human intentions and motivations from visual content. This multi-benchmark evaluation framework allows us to thoroughly assess WindowQuant’s performance across

Table 2. Performance comparison of different video understanding models on the MVBench dataset. "LLaVA-OV" means LLaVA-OneVision-Qwen2. Some of the results are cited from TS-LLaVA [33].

Method	LLM Size	Vision Encoder	AA	AC	AL	AP	AS	CO	CI	EN	ER	UA
Video-ChatGPT	7B	CLIP-L	62.0	30.5	20.0	26.0	23.5	33.0	35.5	29.5	26.0	32.7
Video-LLaMA	7B	CLIP-G	51.0	34.0	22.5	25.5	27.5	40.0	37.0	30.0	29.0	34.1
VideoChat	7B	CLIP-G	56.0	35.0	27.0	26.5	33.5	41.0	36.0	32.5	33.5	35.5
VideoChat2	7B	UMT-L	83.5	39.0	23.0	47.5	66.0	36.5	65.5	35.0	49.5	51.1
ST-LLM	7B	ViT-G	84.0	36.5	31.0	53.5	66.0	46.5	58.5	34.5	44.0	54.9
PLLaVA	7B	CLIP-L	55.5	39.5	26.0	49.0	58.0	53.5	51.0	30.5	41.0	46.6
PLLaVA	34B	CLIP-L	82.0	40.5	49.5	53.0	67.5	66.5	59.0	39.5	47.0	58.1
GPT-4V	GPT4	Unknown	72.0	39.0	40.5	63.5	55.5	52.0	11.0	31.0	46.5	43.5
TS-LLaVA	7B	CLIP-L	58.5	41.0	27.0	53.5	54.0	53.0	32.0	32.5	36.0	45.5
TS-LLaVA	34B	CLIP-L	73.5	46.5	39.0	47.5	61.0	66.5	39.0	42.5	46.5	52.6
LLaVA-OV	7B	SigLIP	72.0	36.0	32.0	40.0	47.9	42.0	41.5	29.5	31.5	60.5
LLaVA-OV+WindowQuant	7B	SigLIP	78.5	38.0	37.0	69.5	66.5	57.5	41.0	31.0	39.5	81.5
Qwen2-VL	7B	ViT-L	81.5	43.0	35.0	42.0	55.3	46.5	59.5	36.0	33.0	73.5
Qwen2-VL+WindowQuant	7B	ViT-L	76.0	42.5	35.5	64	67.0	58.0	48.5	37.5	40.5	74
InternVL2	8B	InternViT	73.5	40.5	38.0	63.0	54.3	51.0	56.0	33.5	37.5	58.5
InternVL2+WindowQuant	8B	InternViT	77.0	35.5	34.5	70.0	69.1	46.5	72.0	33.5	36.5	62.5
Method	LLM Size	Vision Encoder	FA	MA	MC	MD	OE	OI	OS	ST	SC	Avg.
Video-ChatGPT	7B	CLIP-L	22.5	39.5	25.5	23.0	54.0	28.0	40.0	31.0	48.5	32.7
Video-LLaMA	7B	CLIP-G	29.0	32.5	22.5	22.5	48.0	40.5	38.0	43.0	45.5	34.1
VideoChat	7B	CLIP-G	33.5	42.5	20.5	31.5	53.0	40.5	30.0	48.5	46.0	35.5
VideoChat2	7B	UMT-L	49.5	58.5	42.0	23.0	58.0	71.5	42.5	88.5	44.0	51.1
ST-LLM	7B	ViT-G	44.0	78.5	56.5	42.5	80.5	73.5	38.5	86.5	43.0	54.9
PLLaVA	7B	CLIP-L	41.0	52.0	42.0	23.5	56.0	61.0	36.0	82.0	45.0	46.6
PLLaVA	34B	CLIP-L	47.0	70.0	43.0	37.5	68.5	67.5	36.5	91.0	51.5	58.1
GPT-4V	GPT4	Unknown	46.5	22.5	12.0	12.0	18.5	59.0	29.5	83.5	45.0	43.5
TS-LLaVA	7B	CLIP-L	36.0	49.0	30.0	23.5	58.5	59.5	32.0	85.0	43.5	45.5
TS-LLaVA	34B	CLIP-L	46.5	51.5	30.0	24.0	52.5	65.5	37.5	89.5	45.5	52.6
LLaVA-OV	7B	SigLIP	42.5	60.0	38.5	28.5	59.1	54.0	36.0	69.5	42.5	45.5
LLaVA-OV+WindowQuant	7B	SigLIP	44.5	63.5	43.0	28.5	52.5	77.0	36.0	90.5	48.5	53.9
Qwen2-VL	7B	ViT-L	52.0	81.0	63.0	35.0	79.8	53.5	37.0	81.0	45.5	54.4
Qwen2-VL+WindowQuant	7B	ViT-L	45.5	69.0	48.0	33.5	64.1	68.0	34.0	93.0	44.0	54.9
InternVL2	8B	InternViT	40.0	78.0	57.0	43.0	76.3	65.0	32.0	78.0	42.0	53.5
InternVL2+WindowQuant	8B	InternViT	43.0	93.0	56.0	48.0	93.4	70.0	36.0	84.0	44.5	58.2

diverse question-answering scenarios, ranging from temporal reasoning and causal inference to intent recognition and long-form video understanding.

We further use a multiple-task video understanding dataset called MVBench to compare the performance between WindowQuant and SOTA VLMs comprehensively and thoroughly. The benchmark is MVBench, which is a comprehensive video understanding benchmark. We select 19 challenging video tasks from MVBench, containing nine types of reasoning abilities in video understanding. The details of the 19 video tasks in MVBench are as follows: AA (Action Antonym), AC (Action Count), AL (Action Localization), AP (Action Prediction), AS (Action Sequence), CO (Character Order), CI (Counterfactual Inference), EN (Egocentric Navigation), ER (Episodic Reasoning), FA (Fine-grained Action),

Table 3. The performance comparison of WindowQuant with other VLM quantization methods. Some of the results are cited from AKVQ [38].

Model	Method	OE	OI	MA	EN	SC	ST	WQA	TQA	MQA	SQA	DQA	Avg
LLaVA-v1.5-7B	FP16	53.0	46.5	47.5	33.5	36.5	73.0	59.0	47.0	68.5	43.0	45.5	50.3
	RTN (INT4)	46.0	43.5	47.5	28.5	35.5	58.5	43.0	43.5	62.5	41.5	37.5	44.4
	RTN (INT2)	15.5	12.0	12.0	13.5	8.0	7.5	7.0	16.0	13.5	9.5	5.5	10.9
	SmoothQuant	37.5	24.0	24.5	22.0	31.5	24.5	24.0	25.5	26.5	25.5	21.0	26.0
	KIVI	18.0	16.0	24.0	20.0	21.5	16.0	13.0	18.5	27.5	19.0	18.0	19.3
	SKVQ	35.0	39.0	14.0	20.0	40.5	48.5	24.5	20.5	54.5	27.0	13.5	30.6
	WindowQuant	50.5	48.5	49.0	27.5	33.5	65.5	58.5	43.0	67.0	46.5	44.0	48.5
LLaVA-v1.5-13B	FP16	46.0	45.0	50.0	26.5	37.0	70.5	64.0	55.0	74.5	51.0	46.0	51.4
	RTN (INT4)	49.0	42.0	47.5	28.5	35.0	65.5	64.0	51.5	71.0	45.0	47.0	49.6
	RTN (INT2)	27.0	12.0	19.5	11.5	20.0	8.5	6.5	15.0	19.5	14.5	13.0	15.2
	SmoothQuant	36.0	26.5	23.0	21.0	31.0	25.0	24.0	27.0	26.5	25.5	24.5	26.4
	KIVI	28.5	25.5	38.0	26.5	22.0	41.0	34.0	34.0	45.5	29.5	35.5	32.7
	SKVQ	49.0	41.5	40.0	25.5	40.5	45.5	45.5	30.0	48.5	31.5	30.5	38.9
	WindowQuant	47.5	47.0	50.0	26.5	36.5	75.0	65.0	49.5	70.5	53.0	47.0	51.6
LLaVA-v1.6-vicuna-7B	FP16	29.0	18.0	18.5	28.0	20.0	63.0	50.5	41.5	38.5	42.0	41.5	35.5
	RTN (INT4)	33.5	8.5	30.0	18.5	17.0	61.5	27.0	33.0	22.5	27.5	30.5	28.1
	RTN (INT2)	8.0	4.0	3.0	4.5	2.5	4.0	6.5	10.5	3.5	6.5	5.0	5.3
	SmoothQuant	35.0	13.5	23.5	21.5	17.5	26.5	25.5	24.5	28.5	24.5	24.5	24.1
	KIVI	17.5	15.5	27.0	17.5	14.5	35.0	18.5	15.5	21.0	12.5	14.5	19.0
	SKVQ	34.0	26.5	23.5	11.0	32.5	32.0	31.0	11.5	33.0	23.0	23.0	25.5
	WindowQuant	43.0	20.0	19.0	28.0	14.5	56.0	41.5	41.5	30.0	39.0	43.5	34.2
LLaVA-v1.6-mistral-7B	FP16	44.5	46.0	44.0	34.5	38.0	71.0	55.5	49.5	66.0	44.0	38.0	48.3
	RTN (INT4)	35.5	24.0	26.0	28.0	20.0	63.0	50.5	41.5	38.5	42.0	41.5	37.3
	RTN (INT2)	29.0	18.0	18.5	20.0	18.0	11.0	13.5	18.0	25.5	17.5	22.5	19.2
	SmoothQuant	38.5	26.0	26.0	24.0	33.0	45.0	29.0	45.5	27.0	34.0	33.5	32.9
	KIVI	45.5	42.5	47.5	24.0	36.0	53.0	47.0	49.5	62.5	39.0	37.5	44.0
	SKVQ	49.0	36.5	39.5	27.5	34.5	56.0	46.5	47.5	52.5	37.0	33.5	41.8
	WindowQuant	43.0	52.0	55.5	34.0	38.5	70.0	58.5	51.0	66.0	44.5	45.5	50.8

MA (Moving Attribute), MC (Moving Count), MD (Moving Direction), OE (Object Existence), OI (Object Interaction), OS (Object Shuffle), ST (Scene Transition), SC (State Change), UA (Unexpected Action). It is worth noting that the videos in MVBench are generally short, and the evaluation on the MVBench dataset also reflects the capability of WindowQuant on short-video tasks.

We also use a comprehensive benchmark, MileBench [36], to compare the performance of WindowQuant with other quantization methods. We select 12 tasks from MileBench, including Object Existence (OE), Object Interaction (OI), Moving Attribute (MA), Egocentric Navigation (EN), State Change (SC), Scene Transition (ST), Space Understanding (SU), Webpage QA (WQA), Textbook QA (TQA), multimodal QA (MQA), Slide VQA (SQA), and Document QA (DQA).

Baselines. We compare the performance of WindowQuant with several SOTA VLM models in recent years, including Video-LLaVA [22], MovieChat+ [37], Vista-LLaVA [29], DeepStack-L [31], M^3 [2], IG-VLM [18], SF-LLaVA [45], TS-LLaVA [33], LLaVA-OneVision-Qwen2 [20], Qwen2-VL [40], InternVL2 [5], Video-ChatGPT [30], Video-LLaMA [47], VideoLLaMA2 [6], VideoChat [21], ST-LLM [27], PLLaVA [44], GPT-4V [32]. The size of most of the LLMs in these VLMs is 7B, while a few of them are 34B. We also compare the performance of WindowQuant with basic uniform RTN quantization method and some SOTA VLM quantization methods, including SmoothQuant [42], KIVI [28], SKVQ [12].

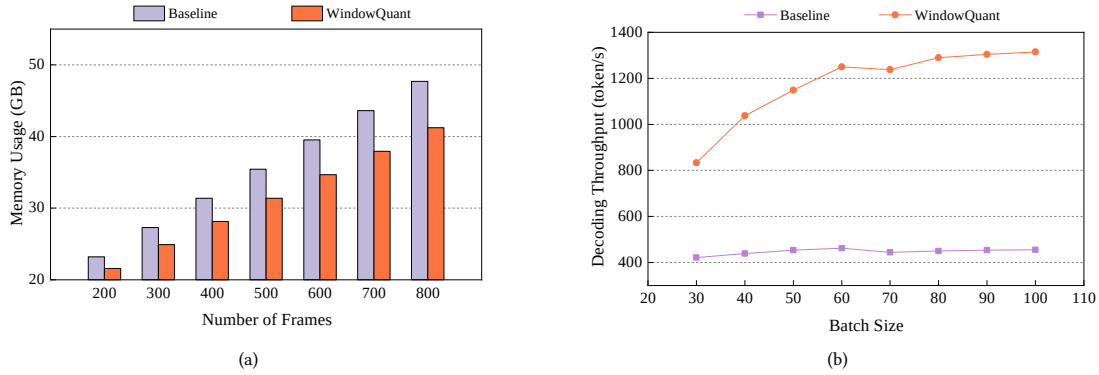


Fig. 10. (a): GPU memory usage comparison of WindowQuant and FP16 model under different number of frames. (b): Decoding throughput comparison of WindowQuant and FP16 model under different batch sizes.

Table 4. GPU memory usage comparison of WindowQuant and other quantization methods.

Method	FP16	RTN (INT4)	RTN (INT2)	SmoothQuant	KIVI	SKVQ	WindowQuant
Average Bit-width	16	4	2	2	2	2	3.17
GPU Memory Usage (GB)	19.13	18.40	18.17	18.17	18.17	18.17	18.33

Table 5. The latency of different processes during inference of FP16 model and WindowQuant.

Method	Input Preparation	Feature Encoding	Similarity Calculation	Prefill Stage	Decoding Stage	Total
FP16	45 ms	920 ms	0 ms	3098 ms	2451 ms	6514 ms
WindowQuant	24 ms	920 ms	31 ms	3099 ms	1640 ms	5714 ms

Implementation. We evaluate WindowQuant on three famous VLM models on the multi-choice QA datasets: LLaVA-OneVision-Qwen2 [20], Qwen2-VL [40], and InternVL2 [5]. Then we implement WindowQuant on four VLM models on the MVBench dataset: LLaVA-v1.5-7B, LLaVA-v1.5-13B, LLaVA-v1.6-vicuna-7B, LLaVA-v1.6-mistral-7B [26]. The original full-precision versions of these models (i.e., the FP16 models) only adopt the standard FlashAttention optimization and do not employ any additional optimization techniques. We conduct all the experiments on an NVIDIA A800 GPU containing 80GB of GPU memory, with a 25-core AMD EPYC 7T83 CPU and 100GB of memory. Unless otherwise specified, in our experiments we generally set the threshold parameter to 2, the window size to 32, the batch size to 1, and extract 100 video frames.

4.2 Main Results

First, we compare the performance of WindowQuant with the baseline VLM models over the three different QA datasets. For the InternVL2 model, we extract 25 video frames in our experiments because its feature extraction code consumes a large amount of GPU memory, and extracting more frames would lead to OOM (out-of-memory) errors. The results are shown in Table 1. Our WindowQuant method demonstrates significant performance improvements across multiple benchmark evaluations. On the NExT-QA and EgoSchema datasets, the models with the WindowQuant

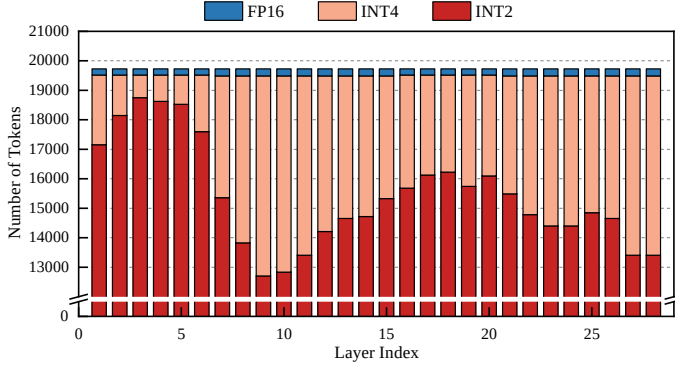
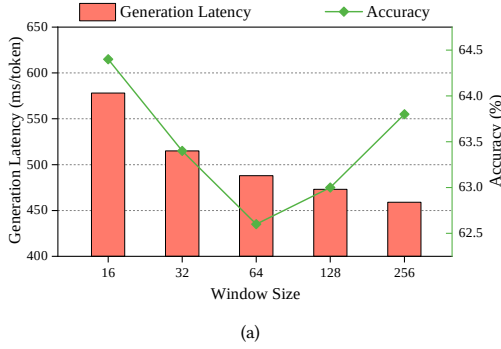


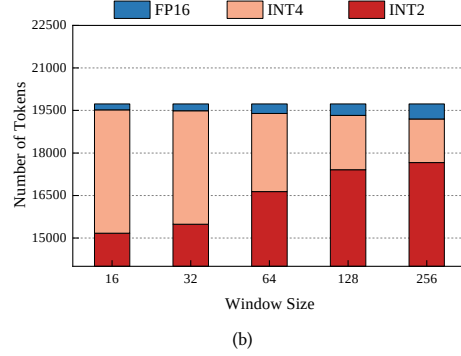
Fig. 11. The number of tokens with different bit-widths in each layer after applying WindowQuant with 100 input frames.

Table 6. The memory usage of different data with 100 input frames.

Data	FP16	WindowQuant
Model weights	15.01 GB	15.01 GB
Encoded features	0.57 GB	0.57 GB
Intermediate tensors in multimodal encoding	1.7 GB	1.7 GB
Intermediate tensors in FFN	2.22 GB	2.22 GB
Intermediate tensors in LayerNorm	0.14 GB	0.14 GB
KV cache	1.05 GB	0.19 GB
Quantization metadata	0 GB	0.06 GB
Output hidden states	0.14 GB	0.14 GB
Peak memory usage	19.13 GB	18.33 GB



(a)



(b)

Fig. 12. (a): The generation latency of WindowQuant under different window sizes. (b): The number of tokens with different bit-widths of WindowQuant under different window sizes.

method achieve the best performance. Specifically, WindowQuant serves as a quantization technique that achieves consistent enhancements when applied to different base models: For the LLaVA-OneVision-Qwen2 model, WindowQuant substantially improves NExT-QA performance from 67.8 to 78.9, EgoSchema from 59.8 to 64.1, and IntentQA from 64.9 to 69.4. When applied to the Qwen2-VL model, WindowQuant shows a slight adjustment in NExT-QA from 75.2 to 74.6, while maintaining competitive performance on IntentQA (88.6 to 87.8). Notably, the InternVL2 model with WindowQuant achieves remarkable improvements: NExT-QA increases from 68.7 to 76.1, EgoSchema from 48.8 to 55.8, and IntentQA from 67.5 to 75.9.

Then, we conduct the performance experiments on the multi-task video understanding dataset MVBench. As illustrated in Table 2, across all sub-tasks, the WindowQuant method universally achieves excellent performance. The highest average accuracy is obtained on the InternVL2 model with WindowQuant. After applying WindowQuant, the LLaVA-OneVision-Qwen2 model achieves notable average performance improvement (59.8 to 64.1), the Qwen2-VL model maintains a competitive average accuracy (54.4 to 54.9), and InternVL2 also delivers an impressive average performance improvement (53.5 to 58.2). These results indicate that WindowQuant, as a quantization optimization strategy, can effectively enhance video understanding task performance with even lower precision data. This is because we use window-level mixed-precision quantization to successfully maintain the precision of those important video

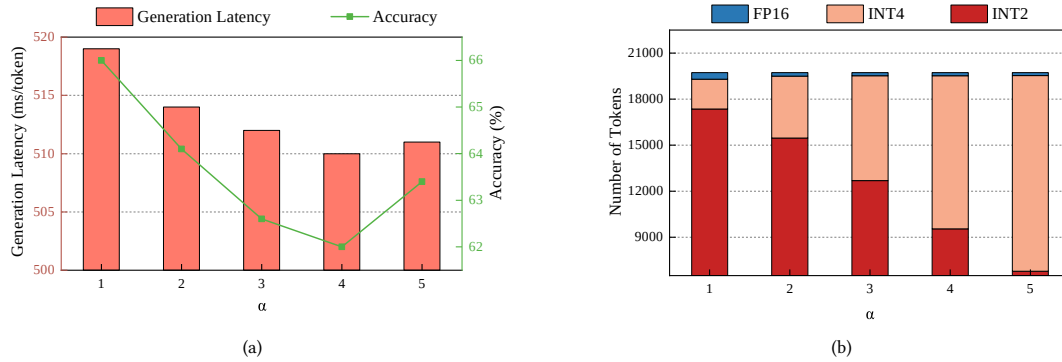


Fig. 13. (a): The generation latency of WindowQuant under different threshold parameter α . (b): The number of tokens with different bit-widths of WindowQuant under different threshold parameters α .

parts. More importantly, since the model’s input contains some redundant information and even noise, our quantization method can appropriately mitigate such information, thereby enabling the model to achieve better performance than the original full precision. Notably, performance degradation is observed on some tasks (e.g. MA, MC, OE), while improvements are achieved on others. This behavior arises from the task-dependent sensitivity to the precision of visual token representations. For datasets that require fine-grained, local, or transient visual information, the group-wise quantization adopted by WindowQuant enforces shared quantization parameters within each window, which reduces the numerical discriminability among tokens. This representation compression amplifies quantization errors in such tasks, leading to degraded model performance. In contrast, for tasks whose inputs span multiple frames and primarily rely on global or redundant temporal cues rather than precise single-token values, the impact of quantization errors is less pronounced. Moreover, WindowQuant effectively suppresses noise and emphasizes stable patterns in these scenarios, resulting in overall performance improvements.

We also compare the performance of WindowQuant with other VLM quantization methods. As illustrated in Table 3, WindowQuant achieves the highest average accuracy over these quantization methods on all four models. On certain models (such as LLaVA-v1.5-13B and LLaVA-1.6-mistral-7B), WindowQuant similarly demonstrates better performance than full precision.

Furthermore, we compare the GPU memory usage and decoding throughput of WindowQuant with the full precision model on the EgoSchema dataset. As shown in Figure 10(a), WindowQuant can reduce the GPU memory usage by 1.61 GB to 6.48 GB compared to the original FP16 precision model as the number of extracted video frames increases. Notably, the decoding throughput means the throughput during the decoding stage only), which excludes the prefill stage and the preceding visual encoder computation. The reason why we choose decoding throughput as an evaluation metric is that, in practical VLM inference, the decoding stage typically dominates the overall runtime, especially for long output sequences and multi-turn dialogue, where a large number of decoding steps are required and the visual encoding and prefill stage computations can be reused across multiple turns. As the number of input video frames increases, the GPU memory usage reduction effect of WindowQuant becomes increasingly significant. This is because a larger number of input frames leads to a larger KV cache, leaving more room for optimization. We evaluate inputs up to 800 frames, as most videos in our selected datasets contain no more than 800 frames. In real-world applications, the number of input frames can be far larger, in which case WindowQuant is expected to achieve even greater memory savings. As for decoding throughput, we extract 5 video frames to avoid OOM error in the experiment. From Figure 10(b)

we can see that WindowQuant significantly improves the decoding throughput of the original model. Moreover, the throughput of the original model quickly reaches its upper limit, whereas the decoding throughput of WindowQuant continues to increase with the batch size and does not reach its limit until the batch size exceeds 100 (the results for batch sizes larger than 100 are not plotted due to GPU memory constraints). Compared with the FP16 baseline, our method reduces GPU memory consumption by approximately 15%, while improving decoding throughput by about 3 $\times$. This substantial decoding throughput gain arises not only from the reduced memory footprint, which shortens the time required to load the KV cache from global memory, but also from the more efficient computation enabled by quantized representations (integer arithmetic is more computationally efficient than floating-point arithmetic.). We also compare the memory usage and average bit-width of WindowQuant and other quantization methods (batch size = 1, number of frames = 100). As shown in Table 4, the memory usage of WindowQuant is lower than RTN (INT4) but higher than other methods. This is because the average bit-width of WindowQuant is about 3.17, while most methods is 2 bit-width (WindowQuant is a mixed-precision quantization method, while other methods are uniform-precision quantization). However, the memory increase is not significant (only 0.16 GB). Considering the improved accuracy of WindowQuant compared to other quantization methods, we believe this increase in memory usage is acceptable.

To investigate which stages of the inference pipeline are optimized by our method, we further compare the latency breakdown of the FP16 baseline and WindowQuant during inference (batch size = 20, number of frames = 5), as summarized in Table 5. In the input preparation stage, WindowQuant achieves a 21 ms reduction in latency compared to FP16, while both methods exhibit identical latency in the feature encoding stage. WindowQuant incurs an additional 31 ms overhead for similarity computation. During the prefill stage, although quantized data in WindowQuant can be loaded and processed more efficiently, the extra quantization operations introduce slight overhead, resulting in a marginal increase of 1 ms in latency. In the decoding stage, WindowQuant reduces latency by 811 ms (in this experiment, we generate 50 tokens for each request; greater reductions can be expected with longer generation lengths). Overall, WindowQuant achieves a total latency reduction of 800 ms.

4.3 Analysis

We apply WindowQuant on Llava-Onevision-Qwen2-7B model and conduct experiments on the EgoSchema dataset to analyze the proportion of tokens with different bit-widths in the KV cache in each layer with 100 input frames after applying WindowQuant. As shown in Figure 11, the number of tokens in INT2 precision is the largest, while that in FP16 precision is the smallest. Moreover, the number of FP16 tokens remains relatively stable across layers. This aligns with our expectations, as only a small portion of visual tokens in each layer is related to the text prompt.

To further study the relationship between token bit-width and memory reduction, we test the main memory usage of different data components during the inference process with 100 input frames. The main memory usage comes from model weights, encoded features, intermediate tensors in visual encoding, intermediate tensors in FFN, intermediate tensors in LayerNorm, KV cache, and quantization metadata. The results are shown in Table 6. Note that the intermediate tensors in vision encoding only appear during the multimodal encoding stage, while the other two types of intermediate tensors appear only during the prefill stage. The peak GPU memory usage occurs during the prefill stage, which does not include the intermediate tensors in vision encoding. The difference between FP16 model and WindowQuant lies in KV cache and quantization metadata. WindowQuant can reduce the memory usage by 0.86 GB with 100 input frames, while introduces a slight cost of quantization metadata of 0.06 GB. In total, WindowQuant can reduce the memory usage of FP16 model by 0.8 GB, corresponds with the results in Figure 10(a). Note that these results are obtained with an input of 100 frames. As shown in Figure 11, the average number of INT2 tokens per layer is 12,564,

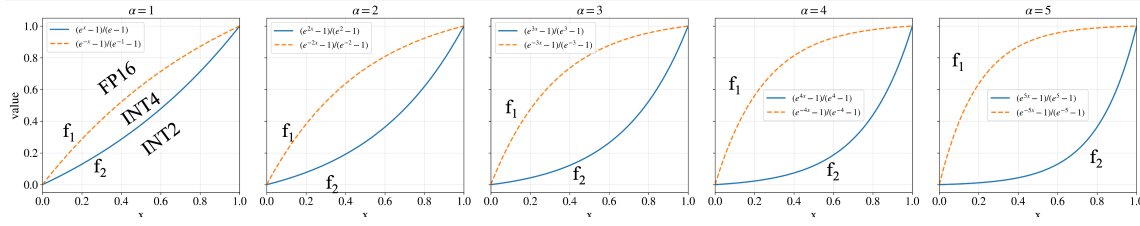


Fig. 14. The function plot of $f_1(x)$ and $f_2(x)$ with different α .

the number of INT4 tokens is 6,956, and only 215 tokens remain in FP16. Accordingly, the average memory reduction per layer is $128 * 4 * 2 * (12564 * \frac{7}{4} + 6956 * \frac{3}{2}) = 33199104 \text{Byte} \approx 31.66 \text{MB}$, resulting in a total memory reduction of $31.66 * 28 = 886.48 \text{MB} \approx 0.86 \text{GB}$ across all layers, which corresponds with the result in Table 6. Note that, since the minimum number of video frames in Figure 10(a) is 200, while the number of video frames tested here is 100, the memory savings observed here differ from those reported in Figure 10(a).

Nevertheless, the intermediate activations in FFN module remain substantial (These data are no longer existing in the decoding stage, but they will still influence peak memory usage), which limits the overall memory reduction proportion achieved by our method. Optimizing the memory footprint of these intermediate activations will be explored in our future work.

We also evaluate the impact of different window sizes on the model. As shown in Figure 12(a), the latency of generating tokens gradually decreases as the window size increases. This is because a larger window size allows for faster quantization, since quantization is performed for all tokens within a window at once. Meanwhile, the model's accuracy first decreases and then increases. To achieve a good balance between latency and accuracy, we choose 32 as the window size value used in our main results. We further examine the distribution of tokens with different precisions under various window sizes, as shown in Figure 12(b). It can be observed that as the window size increases, the numbers of FP16 and INT2 tokens gradually increase, while the number of INT4 tokens gradually decreases. This is because the important video frames within a large window can be surrounded by many unimportant frames, leading to more windows being quantized to INT2. On the other hand, since we fix the precision of the first window as FP16, the number of FP16 tokens will increase slightly as the window size increases.

We further evaluate the model's accuracy and token generation latency under different threshold parameters. As shown in Figure 13(a), as α increases, both the latency and the model's accuracy first decrease and then increase. To achieve a good balance between latency and accuracy, we choose 2 as the threshold parameter value used in our main results. We also analyze the distribution of tokens with different bit-widths under various threshold parameters, as shown in Figure 13(b). As α increases, the numbers of INT2 and FP16 tokens gradually decrease, while the number of INT4 tokens gradually increases. This is consistent with expectations, because the plots of the two threshold functions $f_1(x)$ and $f_2(x)$ (see Equation 10 and Equation 11) show that as α grows, the middle region between the two curves becomes wider (as illustrated in Figure 14), and the tokens falling into this middle region are exactly those that are quantized to INT4.

4.4 Ablation Study

Finally, we conduct an ablation study to evaluate the importance of the three modules in WindowQuant. Table 7 shows the configuration of the three methods we test. "Module I" means the first window with fixed precision. "Module II" means the quantization configuration determination. "Module III" means the KV cache window reordering. Here, "w/o

Table 7. The configuration of each method in the ablation study. “×” means the method doesn’t have that module, while “✓” means the method has that module.

Method	Module I	Module II	Module III
A	×	✓	✓
B	✓	×	✓
C	✓	✓	×
WindowQuant	✓	✓	✓

Table 9. The impact of operator fusion on latency.

Method	Attention Latency	Decoding Latency
Operator Fusion	0.82 ms	1640 ms
w/o Operator Fusion	1.25 ms	2198 ms

Table 8. The accuracy, memory usage, and decoding throughput of different methods in the ablation study.

Method	Accuracy (%)	Memory Usage (GB)	Decoding Throughput (tokens/s)
Full Precision	58.1	47.7	444
A	57.2	43.6	1276
B	55.2	43.7	1250
C	58.4	44.7	500
WindowQuant	58.4	43.7	1250

Table 10. Comparison of two different quantization methods.

Quantization Method	Accuracy	Quantization Metadata
Per-tensor	64.1 %	61.4 MB
Group-wise	58.6 %	0.2 MB

Table 11. Comparison of different similarity functions.

Similarity Function	Latency	Accuracy
Pearson Correlation	55 ms	63.2 %
Euclidean Distance	49 ms	61.4 %
Cosine Similarity	48 ms	64.1 %

Module II” refers to randomly assigning the quantization configuration for each window. We set the batch size to 60, extract 5 frames per video, and test those methods on the EgoSchema datasets with LLaVA-OneVision-Qwen2-7B model. Table 8 shows the ablation study results. From the table, we can see that removing the first-window fixed precision leads to a slight drop in model accuracy, while latency and memory usage are slightly improved. This is because, in this case, the first window may also be quantized, increasing the number of quantized tokens. However, since the first window is crucial, quantizing it reduces the model’s accuracy. Removing the quantization configuration determination results in a more noticeable drop in accuracy, but latency and memory usage remain unchanged. This indicates that our quantization configuration determination indeed plays a key role in identifying important tokens. When KV cache window reordering is removed, the model’s accuracy remains unchanged, which is consistent with our conclusion that KV cache window reordering does not affect accuracy. However, memory usage increases a little and decoding throughput drops significantly because having KV cache segments with mixed precisions in physically contiguous memory reduces hardware efficiency.

We also evaluate the impact of operator fusion on latency, as summarized in Table 9 (batch size = 20, number of frames = 5). With operator fusion enabled, the per-layer attention computation latency for decoding a single token is reduced by 0.43 ms, leading to an overall reduction of 558 ms in total decoding latency.

We further set the batch size to 1 and the number of frames to 100 to evaluate the effects of per-tensor versus group-wise quantization, as well as different similarity computation functions. As shown in Table 10, compared with per-tensor quantization, group-wise quantization slightly increases the memory footprint of quantization metadata (by approximately 61.2 MB, which is negligible compared to the overall memory savings brought by quantization), while achieving an accuracy improvement of about 5.5%. Therefore, we consider group-wise quantization to be highly worthwhile. Table 11 presents a comparison of different similarity computation functions. We evaluate three

functions—pearson correlation, euclidean distance, and cosine similarity—for similarity calculation, and test their computation latency and task accuracy (batch size = 1, number of frames = 100). The results show that cosine similarity achieve the lowest latency and the highest accuracy. Therefore, we choose cosine similarity as our similarity function.

5 Related Works

5.1 KV cache Compression

Past researchers have proposed various methods to compress the KV cache. Some have suggested pruning the attention module within the KV cache [1, 3, 34]. For example, Shazeer [34] proposed MQA, where all attention heads share a single set of KV parameters. Some have proposed evicting unimportant tokens from the KV cache [15, 43, 48]. For instance, Xiao et al. [43] introduced streamingLLM, which retains only the most recent sliding window of KV cache along with the initial tokens. Zhang et al. [48] proposed Q-Hitter, which evaluates the importance of attention based on the sum of attention scores across each token’s column in the attention matrix. Others have approached the problem from a system perspective [7, 8, 19]. For example, Kwon et al. [19] introduced PageAttention, which applies the memory paging concept from traditional operating systems by mapping logically contiguous KV cache pages to physically non-contiguous pages through a page table. Dao et al. [8] proposed FlashAttention, which rewrites certain complex operators to be processed as much as possible within the GPU’s high-speed SRAM. However, these methods are not as easy to implement as quantitative methods, and their optimization results are also not as effective as those achieved by quantitative methods.

5.2 Quantization

Quantization is also an effective method to optimize VLMs’ memory footprint and inference latency. The quantization work initially focused on quantizing weights and activations [9, 10, 13, 14, 23, 42]. For example, Xiao et al. [42] proposed SmoothQuant, which scales weights that are easy to quantize and activations that are difficult to quantize separately. However, during inference with a long input sequence, the KV cache often becomes larger than the weights and activations. Therefore, the quantization work has been extended to the KV cache as well [24, 28, 35, 49]. For instance, Zhao et al. [49] introduced Atom, which performs group quantization of the KV cache to low bits, where each group has independent quantization parameters. Liu et al. [28] proposed KIVI, which applies per-channel quantization to the K cache while keeping the original per-token quantization for the V cache. However, these quantization methods that uniformly quantize tokens to the same bit-width cannot maintain model accuracy effectively. Other researchers have proposed mixed-precision quantization methods to process important tokens [11, 16, 17, 46]. For example, Kim et al. [17] proposed SqueezeLLM, which divides the weights into dense matrices without outliers and sparse matrices containing outliers, then applies low-bit quantization to the dense matrices while keeping the outliers in FP16 precision. Yang et al. [46] proposed MiKV, which identifies unimportant tokens based on attention scores but quantizes them instead of evicting them. Hooper et al. [16] introduced KVQuant, which uses a custom data type called nuqX to represent the mixed-precision quantized KV cache. These methods use token-level quantization search, which is very time-consuming, and they have not adequately addressed the hardware inefficiency issues brought by mixed-precision quantization.

6 Conclusion

This paper introduces WindowQuant, a window-adaptive mixed-precision KV cache quantization method for VLM inference. WindowQuant uses a window-level quantization search module to determine the bit-width configuration of

visual token KV cache windows. It also contains a window-level KV cache computation module, reordering these KV cache windows to avoid hardware inefficiency. Extensive experiments on multiple datasets and models demonstrate that WindowQuant outperforms SOTA VLM models and KV cache quantization methods.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245* (2023).
- [2] Mu Cai, Jianwei Yang, Jianfeng Gao, and Yong Jae Lee. 2024. Matryoshka multimodal models. *arXiv preprint arXiv:2405.17430* (2024).
- [3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774* (2024).
- [4] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*. 578–594.
- [5] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, Jiapeng Luo, Zheng Ma, et al. 2024. How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites. *Science China Information Sciences* 67, 12 (2024), 220101.
- [6] Zesen Cheng, Sicong Leng, Hang Zhang, Yifei Xin, Xin Li, Guanzheng Chen, Yongxin Zhu, Wenqi Zhang, Ziyang Luo, Deli Zhao, et al. 2024. Videollama 2: Advancing spatial-temporal modeling and audio understanding in video-llms. *arXiv preprint arXiv:2406.07476* (2024).
- [7] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).
- [8] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems* 35 (2022), 16344–16359.
- [9] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. *Advances in Neural Information Processing Systems* 35 (2022), 30318–30332.
- [10] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefer, and Dan Alistarh. 2023. Spqr: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078* (2023).
- [11] Shichen Dong, Wen Cheng, Jiayu Qin, and Wei Wang. 2024. QaQ: Quality Adaptive Quantization for LLM KV Cache. *arXiv preprint arXiv:2403.04643* (2024).
- [12] Haojie Duanmu, Zhihang Yuan, Xiuhong Li, Jiangfei Duan, Xingcheng Zhang, and Dahua Lin. 2024. Skvq: Sliding-window key and value cache quantization for large language models. *arXiv preprint arXiv:2405.06219* (2024).
- [13] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).
- [14] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801* (2023).
- [15] Chi Han, Qifan Wang, Wenhan Xiong, Yu Chen, Heng Ji, and Sinong Wang. 2023. Lm-infinite: Simple on-the-fly length generalization for large language models. *arXiv preprint arXiv:2308.16137* (2023).
- [16] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079* (2024).
- [17] Sehoon Kim, Coleman Hooper, Amir Gholami, Zhen Dong, Xiuyu Li, Sheng Shen, Michael W Mahoney, and Kurt Keutzer. 2023. Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629* (2023).
- [18] Wonkyun Kim, Changin Choi, Wonseok Lee, and Wonjong Rhee. 2024. An image grid can be worth a video: Zero-shot video question answering using a vlm. *IEEE Access* (2024).
- [19] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [20] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, et al. 2024. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326* (2024).
- [21] Kunchang Li, Yali Wang, Yanan He, Yizhuo Li, Yi Wang, Yi Liu, Zun Wang, Jilan Xu, Guo Chen, Ping Luo, et al. 2024. Mvbench: A comprehensive multi-modal video understanding benchmark. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 22195–22206.
- [22] Bin Lin, Yang Ye, Bin Zhu, Jiaxi Cui, Munan Ning, Peng Jin, and Li Yuan. 2023. Video-llava: Learning united visual representation by alignment before projection. *arXiv preprint arXiv:2311.10122* (2023).
- [23] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.

- [24] Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. *arXiv preprint arXiv:2405.04532* (2024).
- [25] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual Instruction Tuning. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 34892–34916. https://proceedings.neurips.cc/paper_files/paper/2023/file/6dcf277ea32ce3288914faf369fe6de0-Paper-Conference.pdf
- [26] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *Advances in neural information processing systems* 36 (2023), 34892–34916.
- [27] Ruyang Liu, Chen Li, Haoran Tang, Yixiao Ge, Ying Shan, and Ge Li. 2024. St-llm: Large language models are effective temporal learners. In *European Conference on Computer Vision*. Springer, 1–18.
- [28] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750* (2024).
- [29] Fan Ma, Xiaojie Jin, Heng Wang, Yuchen Xian, Jiashi Feng, and Yi Yang. 2024. Vista-llama: Reducing hallucination in video language models via equal distance to visual tokens. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 13151–13160.
- [30] Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. 2023. Video-chatgpt: Towards detailed video understanding via large vision and language models. *arXiv preprint arXiv:2306.05424* (2023).
- [31] Lingchen Meng, Jianwei Yang, Rui Tian, Xiyang Dai, Zuxuan Wu, Jianfeng Gao, and Yu-Gang Jiang. 2024. Deepstack: Deeply stacking visual tokens is surprisingly simple and effective for llms. *Advances in Neural Information Processing Systems* 37 (2024), 23464–23487.
- [32] OpenAI. 2023. GPT-4V(ision) System Card. https://cdn.openai.com/papers/GPTV_System_Card.pdf. Version 2.
- [33] Tingyu Qu, Mingxiao Li, Tinne Tuytelaars, and Marie-Francine Moens. 2024. TS-LLaVA: Constructing Visual Tokens through Thumbnail-and-Sampling for Training-Free Video Large Language Models. *arXiv preprint arXiv:2411.11066* (2024).
- [34] Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150* (2019).
- [35] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: High-throughput generative inference of large language models with a single gpu. In *International Conference on Machine Learning*. PMLR, 31094–31116.
- [36] Dingjie Song, Shunian Chen, Guiming Hardy Chen, Fei Yu, Xiang Wan, and Benyou Wang. 2024. Milebench: Benchmarking mllms in long context. *arXiv preprint arXiv:2404.18532* (2024).
- [37] Enxin Song, Wenhao Chai, Tian Ye, Jenq-Neng Hwang, Xi Li, and Gaoang Wang. 2025. Moviechat+: Question-aware sparse memory for long video question answering. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [38] Zunhai Su, Wang Shen, Ling Li, Zhe Chen, Hanyu Wei, Huangqi Yu, and Kehong Yuan. 2025. AKVQ-VL: Attention-Aware KV Cache Adaptive 2-Bit Quantization for Vision-Language Models. *arXiv preprint arXiv:2501.15021* (2025).
- [39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [40] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. 2024. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191* (2024).
- [41] Zihao Wang, Bin Cui, and Shaoqun Gan. 2024. SqueezeAttention: 2D Management of KV-Cache in LLM Inference via Layer-wise Optimal Budget. *arXiv:2404.04793* [cs.LG] <https://arxiv.org/abs/2404.04793>
- [42] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*. PMLR, 38087–38099.
- [43] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453* (2023).
- [44] Lin Xu, Yilin Zhao, Daquan Zhou, Zhijie Lin, See Kiong Ng, and Jiashi Feng. 2024. Pllava: Parameter-free llava extension from images to videos for video dense captioning. *arXiv preprint arXiv:2404.16994* (2024).
- [45] Mingze Xu, Mingfei Gao, Zhe Gan, Hong-You Chen, Zhengfeng Lai, Haiming Gang, Kai Kang, and Afshin Dehghan. 2024. Slowfast-llava: A strong training-free baseline for video large language models. *arXiv preprint arXiv:2407.15841* (2024).
- [46] June Yong Yang, Byeongwook Kim, Jeongin Bae, Beomseok Kwon, Gunho Park, Eunho Yang, Se Jung Kwon, and Dongsoo Lee. 2024. No token left behind: Reliable kv cache compression via importance-aware mixed precision quantization. *arXiv preprint arXiv:2402.18096* (2024).
- [47] Hang Zhang, Xin Li, and Lidong Bing. 2023. Video-llama: An instruction-tuned audio-visual language model for video understanding. *arXiv preprint arXiv:2306.02858* (2023).
- [48] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. 2024. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems* 36 (2024).
- [49] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-bit quantization for efficient and accurate llm serving. *Proceedings of Machine Learning and Systems* 6 (2024), 196–209.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009